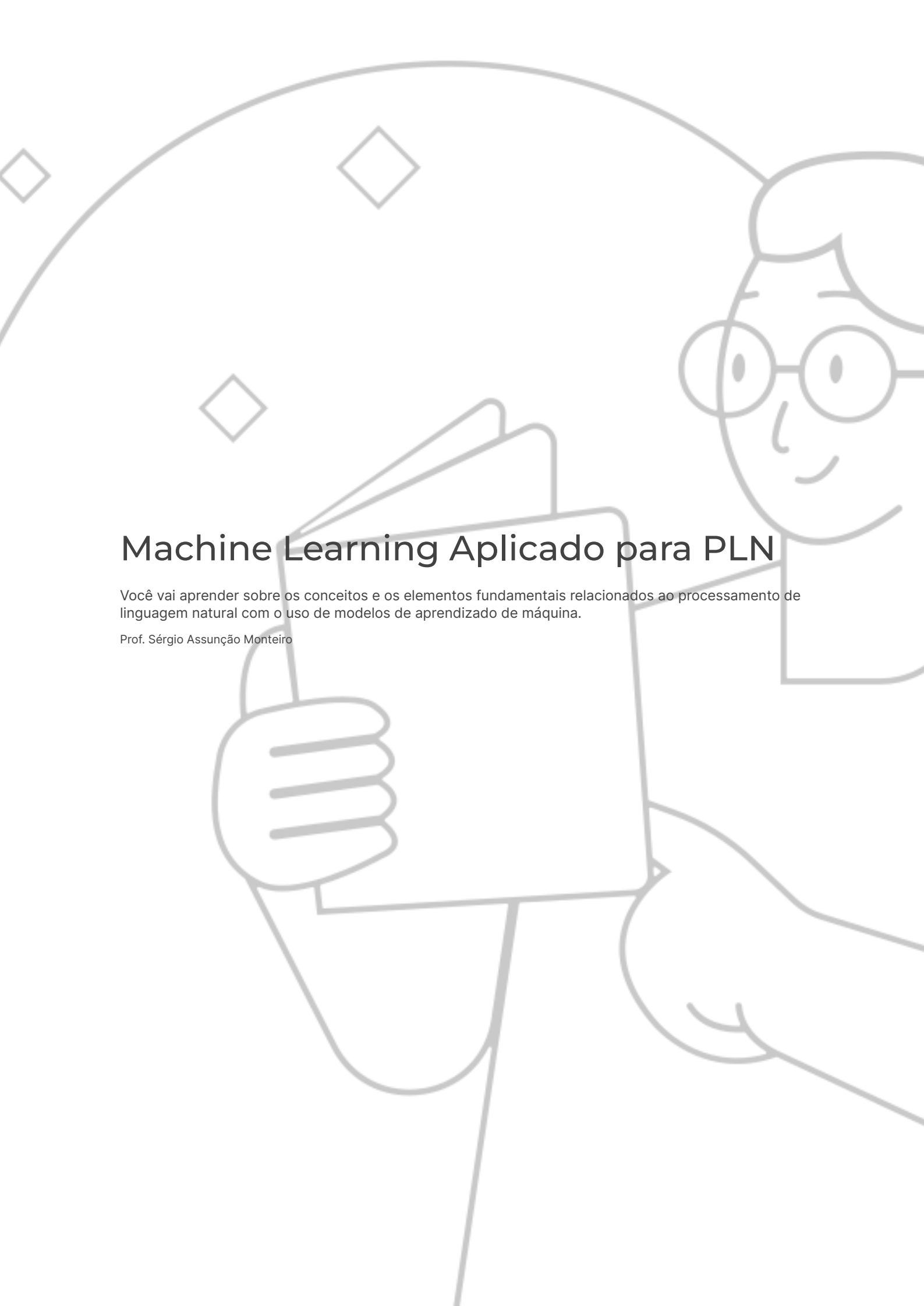




Machine Learning Aplicado para PLN

Você vai aprender sobre os conceitos e os elementos fundamentais relacionados ao processamento de linguagem natural com o uso de modelos de aprendizado de máquina.

Prof. Sérgio Assunção Monteiro

Preparação

Para desenvolver os exemplos na linguagem Python, é importante usar o Google Colab. Ele é gratuito, basta ter uma conta no gmail e praticar os exemplos e exercícios propostos.

Objetivos

- Descrever os conceitos essenciais sobre redes neurais e sua relação com o processamento de linguagem natural.
- Identificar os principais frameworks de computação utilizados para processamento de linguagem natural e como eles são utilizados.
- Aplicar diferentes frameworks para desenvolver aplicações de PLN em Python.

Introdução

Cada vez mais, é necessário utilizar serviços eletrônicos para reduzir despesas financeiras e economizar tempo. A demanda por maior eficiência no gerenciamento do tempo e dos recursos torna ainda mais interessante o investimento no campo da Inteligência Artificial (IA). Não é à toa que uma das áreas em expansão da IA é aquela dedicada ao processamento de linguagem natural (PLN). Por isso, vamos conhecer alguns dos principais modelos de aprendizado de máquina voltados para essa área, a fim de nos situarmos no contexto das aplicações mais modernas que estão revolucionando nossa forma de convivência em sociedade.

Visão geral de redes neurais

Conheça neste vídeo os conceitos essenciais sobre redes neurais e descubra como eles se relacionam com o processamento de linguagem natural.

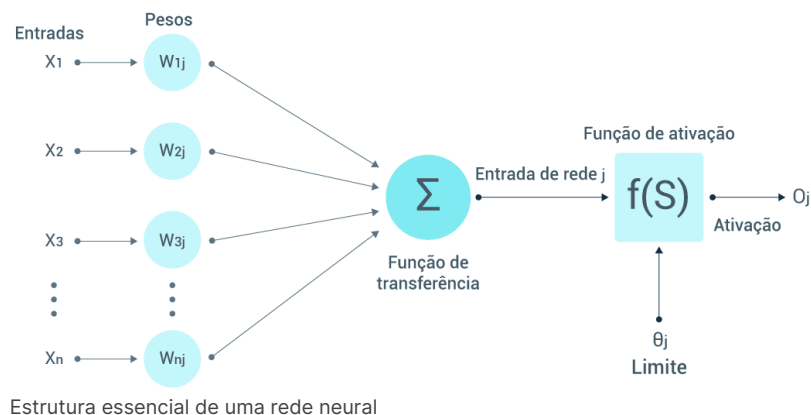


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos essenciais de redes neurais

As redes neurais são modelos computacionais utilizados pela inteligência artificial com inspiração na estrutura e no funcionamento do cérebro humano. Elas são utilizadas em diversas tarefas, como reconhecimento de imagem e processamento de linguagem, fornecendo elementos que servem como suporte para a tomada de decisão. Vamos conferir a seguir uma estrutura básica de uma rede neural.



Ao analisar a imagem, observamos que o modelo é composto por nós interconectados, representando neurônios organizados em camadas. Cada neurônio processa as entradas (*inputs*) aplicando uma função que combina pesos (*weights*) e limites (*threshold*) para produzir uma saída (*activation*). O aprendizado da rede neural ocorre por meio da execução de um algoritmo de otimização que minimiza os erros entre o resultado esperado e o resultado de referência fornecido por um conjunto de dados chamado de **base de treinamento**. Ao final desse processo, as redes neurais são capazes de generalizar características e, assim, aprendem a reconhecer padrões nos dados.

Redes neurais aplicadas para PLN

As redes neurais têm diversas aplicações. Aqui, vamos focar nas aplicações voltadas para processamento de linguagem natural (PLN). De fato, houve um crescimento significativo de aplicações nesse campo, tornando-se cada vez mais comum que pessoas com diferentes níveis de conhecimento usem aplicações nessa área para realizar consultas e estudar sobre os mais diversos assuntos. Parte dessa popularidade se deve à capacidade de as redes neurais aprenderem a reconhecer padrões e representações complexas de grandes quantidades de dados textuais. A seguir, apresentamos alguns modelos de redes neurais que são utilizadas para aplicações de PLN.

Redes neurais recorrentes (RNNs)

São estruturadas para processar dados sequenciais. Exatamente por isso são adequadas para aplicações de PLN, uma vez que possuem elementos estruturais que permitem considerar as entradas anteriores enquanto processam a entrada atual, o que as torna eficazes para entender a natureza sequencial do texto.

Redes neurais convolucionais (CNNs)

São muito utilizadas para tarefas de visão computacional, no entanto, também são aplicadas com sucesso para PLN para fazer classificação de texto e análise de sentimento. Elas se destacam por terem a capacidade de capturar padrões locais e estruturas hierárquicas no texto.

Transformadores

São modelos que utilizam um mecanismo de atenção, que captura as dependências entre as palavras com bastante eficiência. São bastante utilizados em tarefas como resposta a perguntas, classificação de texto e tradução automática.

Transferência de aprendizado e modelos pré-treinados

São modelos baseados na arquitetura dos transformadores utilizados para tarefas de PLN. De forma muito simplificada, esses modelos fazem a geração de texto semelhante ao ser humano por meio da previsão da próxima palavra em uma sequência. São "pré-treinados" em uma grande quantidade de dados de texto. Assim, se tornam mais eficazes para capturar representações contextualizadas de palavras.

Como vimos, as redes neurais têm um papel muito importante nas aplicações de PLN. Adiante, vamos analisar com mais detalhes esses modelos. Agora, vamos fazer um exercício para consolidar nossos conhecimentos. Bons Estudos!

Atividade 1

As redes neurais artificiais ocupam um papel de destaque no processamento de linguagem natural. Curiosamente, alguns modelos de redes neurais são utilizados para mais de um tipo de aplicação como, por exemplo, previsão de séries temporais. Nesse sentido, selecione a opção correta que relaciona aplicações de séries temporais e processamento de linguagem natural.

A

As duas aplicações possuem a mesma natureza numérica.

B

As redes neurais constroem funções matemáticas capazes de determinar exatamente qual será o próximo valor de uma série temporal.

C

As redes neurais associadas à visão computacional são adequadas para fazer previsões de séries temporais.

D

Alguns modelos de redes neurais exploram o processamento sequencial em que as palavras anteriores influenciam as que vão aparecer na sequência.

E

As redes neurais possuem a capacidade de generalizar qualquer tipo de comportamento, seja relacionado a dados numéricos, ou seja, a dados textuais, sem que haja a necessidade de nenhum tipo de adaptação.



A alternativa D está correta.

Os modelos de previsões de séries temporais, de modo geral, utilizam dados históricos e procuram padrões entre eles, de modo que possam fazer estimativas sobre qual será o próximo valor. De fato, o mesmo procedimento ocorre no caso das aplicações de processamento de linguagem natural, no qual as palavras de uma frase possuem dependência entre si.

Redes neurais recorrentes

Explore neste vídeo os conceitos de redes neurais recorrentes e aprenda relacioná-los com o processamento de linguagem natural.

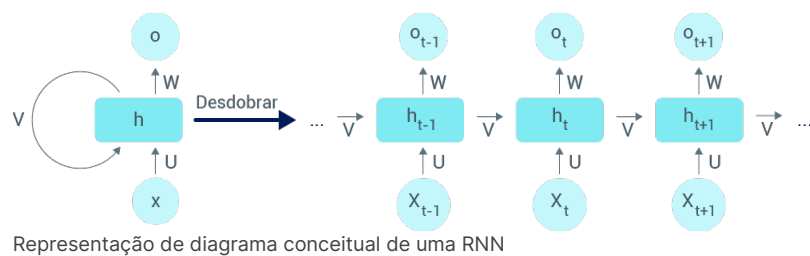


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Estrutura das redes neurais recorrentes

As redes neurais recorrentes (RNNs – *recurrent neural network*) foram projetadas para processar dados sequenciais. Exatamente por causa dessa característica, elas são ideais para aplicações que envolvem previsões de séries temporais, processamento de linguagem natural e de fala. Sua estrutura é composta por conexões que criam *loops*, permitindo que mantenham a memória interna e capturem dependências temporais nos dados de entrada. Vejamos a seguir a estrutura conceitual de uma RNN.



Portanto, na estrutura das RNNs, teremos:

1

Entrada sequencial

As RNNs recebem os dados como uma sequência de entradas, de modo que sejam processados um por um.

2 Memória interna

As RNNs utilizam estados ocultos que funcionam como memória interna, possibilitando guardar informações sobre entradas anteriores na sequência.

3

Parâmetros compartilhados

Os mesmos pesos e vieses são compartilhados em todas as etapas de tempo. Isso faz com que as RNNs aprendam padrões e relacionamentos nos dados sequenciais.

4

Desdobramento ao longo do tempo

O processo de treinamento conecta os estados ocultos em várias etapas de tempo.

5

Backpropagation through time (BPTT)

É uma variação do algoritmo de backpropagation – usado para treinar redes neurais tradicionais. O BPTT é adaptado para tratar a natureza temporal dos dados.

As RNNs se destacam em tarefas em que a ordem e o contexto dos dados são importantes. Vamos nos aprofundar nesse assunto nos próximos passos.

RNNs aplicadas para PLN

As RNNs não se limitam às aplicações de PLN, mas, de fato, elas possuem um grande destaque nessa área. Isso ocorre devido à sua capacidade de lidar com dados sequenciais. Dessa forma, conseguem capturar informações contextuais e dependências entre palavras em frases. Agora, confira algumas das aplicações de RNNs em PLN.

1

Modelagem de linguagem

Prevê a probabilidade da ocorrência de uma palavra dadas as palavras anteriores em uma frase.

2

Geração de texto

Faz a geração de poesias, histórias e diálogos a partir do treinamento do modelo em um grande corpus de texto.

3

Tradução de máquina

Recebe uma frase de entrada em um idioma e a traduz para outro idioma.

4 Análise de sentimento

Classifica o texto em uma das categorias – positivo, negativo, neutro.

5

Reconhecimento de entidade nomeada

Identifica e classifica entidades nomeadas em determinado texto, como nomes, datas e locais, por exemplo.

6

Reconhecimento de fala

Converte a linguagem falada em texto escrito.

7

Resumo de texto

Gera um resumo conciso de um texto mais longo considerando o contexto e informações importantes.

8

Sistemas de diálogo

Podem ser usados para a criação de chatbots que podem interagir com os usuários para gerar respostas apropriadas com base no contexto da conversa.

Como vimos, as RNNs têm tido muito sucesso nas aplicações de PLN. Na sequência, vamos analisar outros modelos de redes neurais aplicadas para processamento de linguagem natural. Agora, é o momento de fazer o exercício para consolidar nossos conhecimentos. Bons Estudos!

Atividade 2

As redes neurais recorrentes têm papel de destaque nas aplicações de processamento natural. Uma dessas aplicações é a de conversação, em que os usuários interagem com um sistema. Nesse sentido, é possível fazer com que dois sistemas baseados em RNNs possam conversar entre si?

A

Sim, uma vez que as RNNs são capazes de se adaptar a qualquer contexto de palavras

B

Sim, desde que os sistemas sejam treinados em bases de dados semelhantes.

C

Sim, pela capacidade das RNNs em processar dados sequenciais.

D

Não, pois as RNNs são preparadas apenas para interagir com usuários humanos.

E

Não, uma vez que as RNNs possuem diversas limitações práticas.



A alternativa B está correta.

Para que as RNNs possam produzir respostas e perguntas que façam sentido, elas precisam ser submetidas a um processo de treinamento que esteja contextualizado na área de atuação delas. Então, se essa condição for satisfeita, podemos colocar dois sistemas baseados em RNNs para interagir entre si.

Redes neurais de convolução

Explore no vídeo os conceitos das redes neurais convolucionais, estabelecendo conexões com o processamento de linguagem natural.

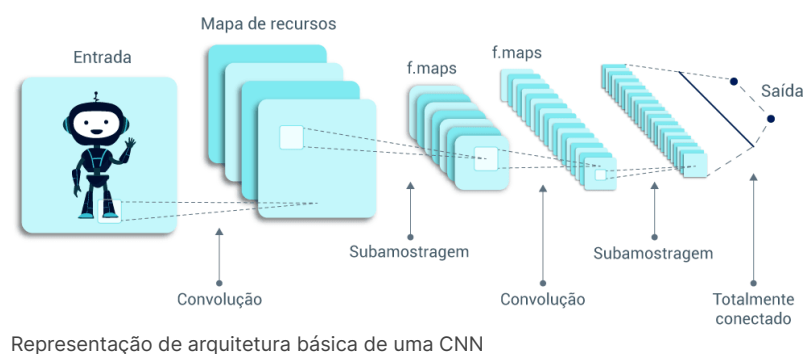


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos essenciais de redes neurais de convolução (CNN)

As CNNs (*convolutional neural network*) são modelos de aprendizado profundo (*deep learning*), que foram projetadas, inicialmente, para aplicações de reconhecimento visual, como classificação de imagens, detecção de objetos e segmentação de imagens. Confira a arquitetura básica de uma CNN.



Representação de arquitetura básica de uma CNN

Na sequência, veremos as camadas essenciais da arquitetura de uma CNN. Mas antes, é importante fazer uma observação. Muitos textos utilizam o termo **recursos** em vez de características, no entanto, ambos possuem o mesmo significado e optamos por usar o termo **características**. Observe tais camadas!

1

Camadas de convolução

É uma técnica utilizada para capturar características de uma fonte de dados e criar um mapa de características por meio da aplicação de filtros (kernels).

2 Camada de pooling
Faz a redução do mapa de características.

3
Funções de ativação
Tem como objetivo aprender os padrões complexos dos dados.

4
Camadas totalmente conectadas
Fazem a conexão de todos os neurônios da camada anterior à camada de saída.

5
Dropout
É uma técnica que tem como objetivo evitar que o modelo fique superespecializado em uma base de treinamento. Esse efeito é chamado de *overfitting*. A ideia é descartar neurônios de modo aleatório para aumentar as chances de reconhecerem diferentes entradas.

Apesar de terem muito sucesso no campo da visão computacional, as CNNs também são usadas no processamento de linguagem natural e reconhecimento de fala, como veremos a seguir.

CNNs aplicadas para PLN

As CNNs podem ser aplicadas ao processamento de linguagem natural. No entanto, surge a pergunta: o que visão computacional e processamento de linguagem natural têm em comum?

Vamos pensar um pouco em termos de processamento de dados: para tratar uma imagem, precisamos convertê-la para uma matriz numérica. No caso do texto, precisamos converter palavras em vetores numéricos. Assim, as CNNs passam a trabalhar com uma dimensão em vez de duas ou três.

De modo semelhante a outras redes neurais de aprendizado profundo aplicadas para processamento de linguagem natural, as CNNs podem ser utilizadas para desempenhar as seguintes tarefas.

- Classificação de texto.
- Classificação de documentos.
- Reconhecimento de entidades nomeadas.
- Geração de texto.
- Modelagem de linguagem.

Nesse último tipo de aplicação, as CNNs são usadas como parte de arquiteturas mais complexas.

Acabamos de conhecer mais uma rede neural usada no processamento de linguagem natural. Mais adiante, veremos outras arquiteturas mais elaboradas, e que têm sido utilizadas com muito sucesso atualmente. No entanto, antes de prosseguir, realizaremos mais um exercício para solidificar o que aprendemos até agora.

Atividade 3

As redes neurais de convolução (CNNs) têm sido aplicadas com bastante sucesso para tarefas de visão computacional. No entanto, sabemos que elas também são utilizadas para processamento de linguagem natural. Nesse sentido, selecione a opção correta que contém o principal benefício do uso das CNNs para processamento de linguagem natural.

A

É a única rede neural capaz de reconhecer entidades nomeadas.

B

Tem a capacidade de reduzir o tamanho de um vocabulário.

C

Faz o tratamento eficiente de dados estruturados.

D

Não precisa de nenhum tipo de adaptação para tratar dados textuais e de imagens.

E

Captura características relevantes em sequências de textos.



A alternativa E está correta.

O processamento de linguagem natural se assemelha a um problema de séries temporais, uma vez que as sequências das palavras são relevantes para que possamos ter uma compreensão do significado do texto como um todo. Nesse sentido, as CNNs são bastante interessantes para esse tipo de aplicação, uma vez que capturam as relações entre os dados.

Modelo Transformer

Assista ao vídeo para receber uma introdução ao modelo Transformer e descubra por que ele é essencial nas aplicações de processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

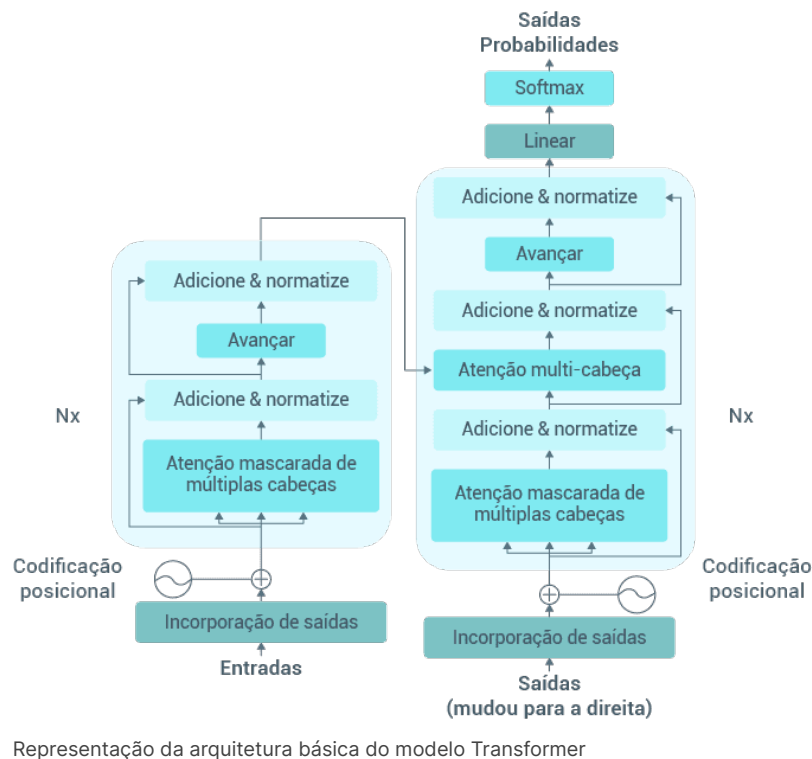
Arquitetura e aspectos do modelo Transformer

Esse modelo é uma das mais modernas arquiteturas de redes neurais aplicada para processamento de linguagem natural e foi introduzido no artigo *Attention is All You Need*, de Vaswani *et al.* (2017). Sua característica central é o mecanismo de atenção, que permite ao modelo focalizar os aspectos essenciais de uma sequência de entrada. Confira uma representação básica (ou nem tão básica assim) do modelo Transformer.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Aqui estão alguns pontos que precisamos destacar no modelo Transformer, veja!

Arquitetura sem recorrência

O modelo Transformer não possui loops. Isso permite explorar a paralelização do processo de treinamento, sendo uma grande diferença em relação aos modelos RNNs.

Codificador e decodificador

O modelo Transformer utiliza a camada de codificação para processar uma sequência de entrada com o objetivo de gerar representações contextuais. Já o decodificador utiliza essas representações para gerar a sequência de saída.

Conexões residuais e normalização

O objetivo é reduzir problemas de aprendizado do modelo. Em especial, minimizar os impactos de um problema conhecido como desaparecimento do gradiente, durante o treinamento.

Um dos principais modelos do tipo Transformers é o BERT (*bidirectional encoder representations from transformers* – Representações de Codificador Bidirecional de Transformadores). Esses modelos são muito populares por produzirem resultados de altíssima qualidade. São pré-treinados com grandes volumes de texto e, depois, ajustados para tratar tarefas específicas. Posteriormente, vamos conhecer um pouco mais sobre as aplicações do modelo Transformer para processamento de linguagem natural.

Modelo Transformers aplicado para PLN

O modelo Transformers é utilizado em diversas aplicações de PLN. Acompanhe!

1 Modelagem de linguagem

O modelo é treinado para prever a probabilidade da ocorrência de uma palavra dadas as palavras anteriores e posteriores.

2

Transferência de aprendizado

o modelo Transformer é pré-treinado em volumes de texto – conhecidos como corpora ou corpus – de maneira não supervisionada. Assim, o modelo consegue capturar as relações entre as palavras.

3

Ajuste fino para tarefas específicas

Após a fase de pré-treinamento, o modelo pode ser adaptado para tarefas específicas, como análise de sentimento, respostas a perguntas, reconhecimento de entidades nomeadas e tradução automática.

4

Captura eficiente de dependências

O mecanismo de atenção do Transformer faz com que ele dê foco nas palavras relevantes de uma frase.

O sucesso do modelo Transformer está na capacidade de aprender padrões e relacionamentos complexos em linguagem natural. Além de ter o modelo BERT, o modelo GPT também é uma aplicação do modelo Transformer. Mais adiante, vamos estudá-lo com maiores detalhes.

Atividade 4

O modelo Transformer tem tido muito destaque em relação a outros modelos, como os de redes neurais recorrentes (RNNs), devido à capacidade de fornecer excelentes resultados nas aplicações de processamento de linguagem natural. A principal característica do modelo Transformer é o uso do mecanismo de Atenção. Nesse sentido, selecione a opção correta que apresenta uma vantagem do modelo Transformer aplicado ao processamento de linguagem natural, quando comparado com RNNs?

A

A arquitetura dos Transformers permite processar sequências de comprimento variável de forma paralela.

B

Os Transformers são muito mais eficientes computacionalmente do que as RNNs.

C

Os Transformers não fazem pré-processamento de sequência de entrada de dados.

D

Os RNNs utilizam mais dados de treinamento do que os Transformers.

E

Diferente das RNNs, os Transformers podem ser aplicados para modelagem de linguagem natural.



A alternativa A está correta.

As RNNs utilizam uma arquitetura sequencial que torna a tarefa de treinar em sequências longas em um grande desafio. Já no caso do modelo Transformer, o uso dos mecanismos de atenção permite capturar as relações de dependência entre todas as palavras da sequência, possibilitando, assim, que processem e capturem informações relevantes de maneira mais eficiente para diferentes tamanhos de sequências.

Transformador generativo pré-treinado

Assista ao vídeo para compreender os conceitos do transformador pré-treinado generativo e sua aplicação no processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

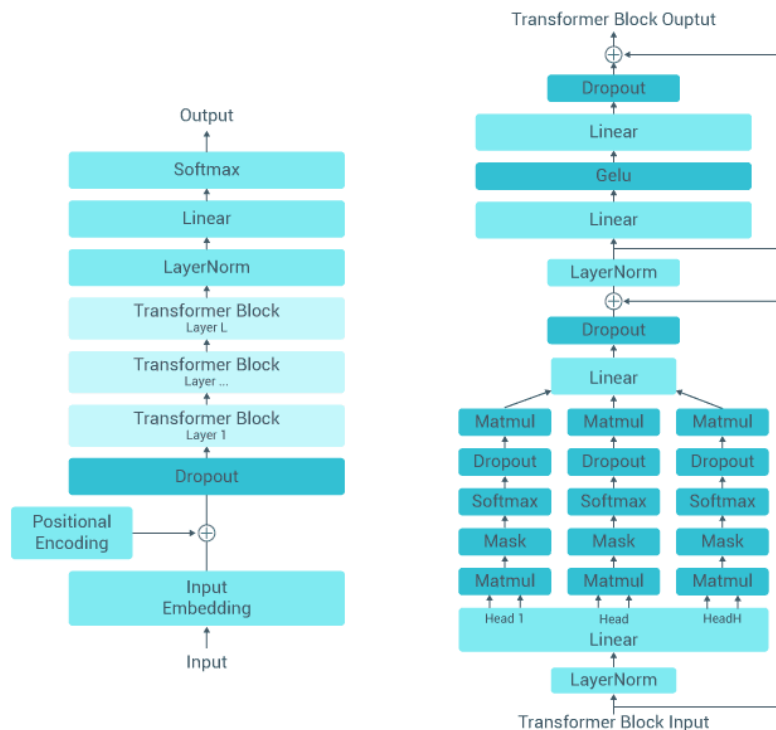
Estrutura e características do transformador generativo pré-treinado

O transformador generativo pré-treinado (GPT) é um modelo de linguagem desenvolvido pela OpenAI. Ele é baseado nos modelos Transformers, que estudamos anteriormente. Não há dúvidas que o GPT se trata de uma grande revolução no processamento de linguagem natural, com aplicações para todas as áreas de conhecimento, que ganhou grande notoriedade no ano de 2023. A seguir, veja a arquitetura básica (ou talvez não tão básica) do modelo GPT.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Demonstração do modelo GPT

Agora, conheça os aspectos essenciais do transformador pré-treinado generativo (GPT).

1

Arquitetura Transformer

O GPT é baseado na arquitetura do modelo Transformer, ou seja, utiliza mecanismos de atenção.

2

Modelo de linguagem generativa

Significa que o modelo pode gerar texto semelhante ao que é produzido por um humano.

3

Pré-treinamento com aprendizado não supervisionado

O GPT utiliza um grande volume de dados de texto com aprendizado não supervisionado. Na fase de pré-treinamento, o modelo constrói conexões, ou seja, aprende a prever a próxima palavra em uma sequência, com base em seu contexto.

4

Mecanismo de atenção

É por meio desse mecanismo que o modelo GPT captura dependências entre palavras em uma frase e estabelece a importância delas com base no contexto.

5 Geração autorregressiva

Trata-se de uma abordagem de previsão de séries temporais em que as palavras anteriores são utilizadas para gerar sentenças coerentes.

6

Avanço contínuo

O modelo GPT tem variantes, como GPT-2 e GPT-3 que demonstraram a evolução para aumentar a capacidade de compreensão, geração e processamento de texto em linguagem natural.

De fato, o GPT tem produzido resultados impressionantes, que vão influenciar a forma como trabalhamos e estudamos. Na sequência, vamos detalhar mais algumas características do modelo GPT.

Transformador generativo pré-treinado aplicado para PLN

Os transformadores generativos pré-treinados causaram um grande impacto nas aplicações de processamento de linguagem natural, devido à capacidade de compreender e gerar texto de forma semelhante aos humanos. Sua atuação tem duas etapas que se destacam: pré-treinamento e ajuste fino. Vamos conhecê-las!

Pré-treinamento

É a etapa onde o transformador generativo é treinado com grande *corpus* de dados de texto de maneira não supervisionada. Assim, o modelo desenvolve a capacidade de relacionar as palavras e pode fazer previsões sobre a próxima palavra em uma sequência com base em seu contexto. Desse modo, o texto resultante passa a ter uma estrutura sintática e semântica coerente. Nesse processo, são utilizados conjuntos de dados que envolvem bilhões de palavras.

Ajuste fino

É a etapa em que precisamos ajustar o modelo para realizar tarefas específicas, visto que a etapa de pré-treinamento capacita o modelo a fazer conexões complexas que, no entanto, ainda são muito gerais. Esse processo modifica alguns dos parâmetros do modelo para que fiquem mais coerentes com o contexto que vai realizar a tarefa.

As aplicações de transformadores generativos pré-treinados

O modelo GPT tem diversas aplicações práticas envolvidas. Confira!

1

Geração de texto

A partir de uma sequência inicial de palavras, o modelo pode produzir histórias e poesias, além de dialogar com o interlocutor.

2

Conclusão e sugestão de texto

O modelo é capaz de recomendar sugestões para completar frases com base em uma entrada parcial.

3 Respostas a perguntas

O GPT pode produzir respostas a perguntas com base em determinado contexto.

4

Sumarização de texto

O GPT é capaz de resumir textos.

5

Tradução de idiomas

O modelo pode converter textos de um idioma para outro.

Essas são apenas algumas das aplicações nas quais podemos utilizar o modelo GPT. Essa área vem atravessando uma fase muito interessante, que irá criar muitas oportunidades de trabalho e de pesquisa. Mais adiante, vamos conhecer alguns dos frameworks que nos permitem utilizar os modelos de aprendizado de máquina para trabalhar com processamento de linguagem natural. Porém, antes de avançar, vamos fazer mais um exercício para consolidar o nosso conhecimento.

Atividade 5

Os transformadores generativos pré-treinados (GPT) têm revolucionado a forma como interagimos com as aplicações de Inteligência Artificial. A qualidade dos resultados que eles fornecem tem atraído cada vez mais usuários que os procuram com os mais diversos interesses. Nesse sentido, selecione a afirmação verdadeira sobre os modelos GPT.

A

Esses modelos são treinados exclusivamente por meio de aprendizado supervisionado.

B

A arquitetura dos transformadores generativos é extremamente simples.

C

Os modelos GPT precisam passar por uma etapa de adequação para produzir resultados coerentes.

D

A fase de pré-treinamento é importante para o aprendizado dos modelos GPT, mas não é essencial.

E

Os modelos GPT podem aprender de forma eficiente com poucos dados.



A alternativa C está correta.

Os transformadores generativos pré-treinados são modelos de linguagem que precisam passar pela fase de pré-treinamento em um grande volume de *corpus* de texto. Em seguida, devem ser ajustados para trabalharem em tarefas específicas.

Visão geral sobre frameworks

Assista ao vídeo e descubra o conceito de frameworks de computação e como eles desempenham um papel fundamental no processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos gerais sobre frameworks

Frameworks são usados em diversos contextos para indicar estruturas com predefinição de como devemos trabalhar. Na computação, eles orientam o modo como desenvolvemos uma aplicação, sendo extremamente úteis na prática ao padronizar o processo de desenvolvimento de software por meio de um conjunto de ferramentas e bibliotecas. Explore agora alguns aspectos essenciais que um framework de software deve ter.

1

Definição e propósito

Trata-se do objetivo para qual o framework foi desenvolvido. Por exemplo, podemos trabalhar com frameworks para desenvolvimento de aplicações de aprendizado de máquina.

2

Abstração e modularidade

Significa que o framework deve apresentar uma camada de alto nível que auxilie o desenvolvedor a abstrair complexidades de uma tecnologia. Por exemplo, essa abstração pode ser obtida por meio de componentes de software que têm objetivos específicos e que podem ser testados separadamente.

3

Componentes reutilizáveis

São elementos como bibliotecas, módulos e APIs, que os frameworks oferecem para implementar funcionalidades comuns. Um exemplo são os frameworks que oferecem modelos de aprendizado de máquina que podem ser utilizados pelo desenvolvedor.

4

Suporte para linguagens de programação

Um framework não é uma linguagem de programação, de forma que deve oferecer recursos para que possa ser utilizado em uma ou mais linguagens de programação. Um exemplo disso são os frameworks suportados por linguagens de programação, como Python e JavaScript.

Além dos pontos mencionados, é esperado que um framework tenha uma boa documentação para facilitar o estudo. Apesar das vantagens que citamos, um framework impõe restrições ao desenvolvimento de software. O jargão comum "engessar o programador" é muito utilizado para se referir às imposições que o framework faz ao desenvolvedor. No entanto, é inegável a importância dos frameworks para padronizar e acelerar o processo de criação de aplicações.

Tipos de frameworks

Um framework não deve ser confundido com uma biblioteca. As principais diferenças entre os dois são o **propósito** e o **escopo**. No caso do framework, tratamos de aspectos mais gerais para a criação de uma aplicação, enquanto a biblioteca tem uma abrangência mais específica. Podemos encontrar frameworks com diferentes propósitos, veja!

1

Aplicações web

Esses frameworks fornecem recursos para solicitações HTTP, roteamento, modelagem e interações de banco de dados. Alguns exemplos desse tipo de framework são: Django (Python), Flask (Python), Ruby on Rails (Ruby), Express.js (JavaScript) e ASP.NET (C#).

2

Interface do usuário

Esses frameworks fornecem componentes e estilos pré-construídos para a construção de interfaces de usuário. Alguns exemplos são o NextJS (JavaScript/TypeScript) e Angular (JavaScript/TypeScript).

3

Tratamentos de testes

Esses frameworks oferecem ferramentas e APIs para a realização de testes automatizados. Alguns exemplos são o JUnit (Java), pytest (Python) e Jasmine (JavaScript).

4

Ciência de dados e aprendizado de máquina

Esses frameworks oferecem ferramentas para manipulação, análise, visualização de dados e modelos de aprendizado de máquina. Especificamente no caso de processamento de linguagem natural, vamos abordar os seguintes frameworks: **spaCy**, **gensim**, **scikit-learn** e **Transformers**.

Mais adiante, vamos abordar os frameworks que mencionamos para processamento de linguagem natural. Agora, chegou o momento de fazermos mais um exercício para consolidar nossos conhecimentos.

Atividade 1

Os frameworks têm diversos usos no desenvolvimento de software. Apesar do propósito geral deles, podemos classificá-los em categorias. Nesse sentido, selecione a opção correta que generaliza os tipos de frameworks de computação.

A

Disponibilizar módulos de aprendizado de máquina.

B

Compartilhar testes automatizados.

C

Oferecer módulos de interface de usuário.

D

Oferecer componentes reutilizáveis.

E

Possuir módulos de gerenciamento de tabelas de banco de dados.



A alternativa C está correta.

Os frameworks são úteis como estruturas para padronização de desenvolvimento de aplicações. Nesse sentido, é essencial que possuam componentes reutilizáveis que possam ser oferecidos para criar aplicações com diferentes finalidades.

Framework spaCy

Assista ao vídeo para explorar os principais componentes do framework spaCy, destacando suas principais funcionalidades voltadas para o processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos conceituais sobre o framework spaCy

O spaCy é um framework de processamento de linguagem natural de código aberto, desenvolvido em Python pela equipe da Explosion AI. Para mais informações, você pode visitar o site oficial em <https://spacy.io/>.

Agora, vamos conferir os motivos que tornaram o spaCy popular.

Eficiência

Utiliza estruturas de dados e algoritmos, que o tornam adequado para processar grandes volumes de dados de texto.

Personalização

Permite que os desenvolvedores façam ajustes nos modelos de acordo com o domínio de suas aplicações.

Facilidade de uso

Fornece componentes pré-construídos para diversas tarefas que são comuns no processamento de linguagem natural.

Precisão

Oferece modelos de aprendizado de máquina pré-treinados.

Suporte multilíngue

Oferece suporte para vários idiomas.

Na sequência, vamos conhecer algumas das principais funcionalidades do spaCy voltadas para processamento de linguagem natural.

Principais funcionalidades de PLN do framework spaCy

O spaCy oferece diversos recursos para processamento de linguagem natural. Agora, vamos apresentar alguns desses recursos, seguidos de exemplos.

Tokenização

Tem como objetivo dividir texto em tokens individuais. O trecho de código a seguir apresenta como utilizarmos a tokenização no spaCy.

```
python

nlp = spacy.load("en_core_web_sm")
texto = "Eu uso o framework spaCy."
doc = nlp(texto)
for token in doc:
    print(token.texto)
```

Marcação de parte do discurso

Tem como objetivo atribuir classes gramaticais do discurso a cada token. O trecho de código a seguir, apresenta como utilizarmos a marcação de parte do discurso no spaCy.

```
python

for token in doc:
    print(token.text, token.pos_)
```

Reconhecimento de entidade nomeada

Tem como objetivo identificar e classificar entidades nomeadas. O trecho de código adiante apresenta como utilizarmos o reconhecimento de entidade nomeada no spaCy.

```
python

for ent in doc.ents:
    print(ent.text, ent.label_)
```

Esses são apenas alguns dos exemplos do que podemos fazer com o spaCy. Para conhecer com mais profundidade como o framework funciona, visite o site oficial e estude os seus recursos.

Atividade 2

O framework spaCy oferece diversas facilidades para o desenvolvimento de aplicações de processamento de linguagem natural. Dessa forma, ele é uma opção adequada para criarmos aplicações nessa área. Nesse sentido, selecione a opção que apresenta corretamente o principal objetivo do framework spaCy para o desenvolvimento de aplicações de processamento de linguagem natural.

A

Gerar texto para treinamento de modelos de linguagem.

B

Criar interfaces de usuário interativas para aplicações web.

C

Analisar o sentimento das postagens nas redes sociais.

D

Realizar processamento eficiente e com precisão de textos.

E

Converter a linguagem falada em texto escrito.



A alternativa D está correta.

O framework spaCy foi desenvolvido para realizar o processamento eficiente e preciso de dados de textos. Ele consegue alcançar esses objetivos por meio de algoritmos e estruturas de dados eficientes.

Framework Gensim

Explore no vídeo os principais aspectos do framework Gensim e aprenda as suas principais funcionalidades no contexto do processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos conceituais sobre o framework Gensim

Gensim é um framework de código aberto para processamento de linguagem natural desenvolvido em Python. É ideal para lidar com grandes coleções de texto e foi projetado especialmente para tarefas como vetorização de texto, cálculo de similaridade de documentos, modelagem de tópicos e outras tarefas de análise de texto. Para obter mais informações e acesso ao Gensim, visite o site em <https://radimrehurek.com/gensim/>.

Agora, conheça as características do Gensim que permitem uma análise de texto eficiente e eficaz.

1

Modelagem de tópicos

Implementa algoritmos como análise semântica latente (LSA) e Alocação de Dirichlet latente (LDA) para descobrir os tópicos em uma coleção de documentos sem a necessidade de rotulagem manual.

2

Incorporações de palavras

Fornecer ferramentas para fazer representações vetoriais de palavras, como os algoritmos como Word2Vec e FastText.

3

Similaridade de documentos

Oferece recursos para calcular a similaridade entre documentos com base em suas representações vetoriais.

4

Vetorização de texto

Oferece suporte para vários métodos de transformação de texto em representações numéricas, que podem ser usadas em modelos de aprendizado de máquina.

Em seguida, vamos conhecer algumas das principais funcionalidades do Gensim voltadas para processamento de linguagem natural.

Principais funcionalidades de PLN do framework Gensim

O Gensim oferece diversos recursos para processamento de linguagem natural. Vejamos alguns desses recursos, seguidos de exemplos.

Incorporações de palavras com Word2Vec

Tem como objetivo principal criar representações numéricas de palavras a partir de dados de texto usando o algoritmo Word2Vec. O trecho de código a seguir apresenta como utilizarmos a incorporação de palavras no Gensim.

python

```
from gensim.models import Word2Vec
frases = [["laranja", "é", "uma", "fruta"], ["banana", "também", "é", "uma", "fruta"]]
modelo = Word2Vec(frases, vector_size=100, window=5, min_count=1, sg=0)
vetor = modelo.wv["laranja"]
```

Similaridade de documentos com consultas de similaridade

Tem como objetivo calcular a semelhança entre documentos usando incorporações de palavras. O trecho de código adiante apresenta como utilizarmos a similaridade de documentos no Gensim.

python

```
from gensim.models import Word2Vec
from gensim.similarities import SoftCosineSimilarity
modelo = Word2Vec.load("word2vec.model")
indice = SoftCosineSimilarity(modelo.wv)
semelhanca = indice[["laranja é uma fruta"], ["banana também é uma fruta"]]
```

Modelagem de tópicos com alocação de Dirichlet latente (LDA)

Tem como principal objetivo realizar a modelagem de tópicos utilizando o algoritmo LDA. O próximo trecho de código apresenta como utilizarmos a modelagem de tópicos com o algoritmo LDA.

python

```
from gensim.models import LdaModel
from gensim.corpora import Dictionary
texto = [["laranja", "é", "uma", "fruta"], ["banana", "também", "é", "uma", "fruta"]]
dicionario = Dictionary(texto)
corpus = [dicionario.doc2bow(palavra) for palavra in texto]
modelo = LdaModel(corpus, num_topics=2, id2word=dicionario)
```

Vimos apenas alguns exemplos do que podemos fazer com um framework voltado para PLN. Para conhecer com mais profundidade como o Gensim, precisamos visitar o site oficial e estudar os recursos disponíveis. Agora, chegou a hora de fazermos mais um exercício para consolidar nossos conhecimentos.

Atividade 3

O framework Gensim é bastante elaborado, e oferece vários algoritmos para o desenvolvimento de aplicações de processamento de linguagem natural. Essas funcionalidades são essenciais para escrever aplicações eficientes e padronizadas. Nesse sentido, selecione a opção que apresente corretamente o principal objetivo do framework Gensim para o desenvolvimento de aplicações de processamento de linguagem natural.

A

Fazer a modelagem de tópicos e incorporação de palavras.

B

Analisar o sentimento em textos nas páginas web.

C

Construir modelos de aprendizagem profunda para classificação de imagens.

D

Criar interfaces de usuário interativas para aplicações web.

E

Traduzir idiomas.



A alternativa A está correta.

O framework Gensim foi projetado com a finalidade de tratar tarefas de processamento de linguagem natural com foco na modelagem de tópicos e na criação de incorporações de palavras.

Frameworks Scikit-Learn e Tensorflow

Assista ao vídeo e confira os principais aspectos dos frameworks Scikit-Learn e Tensorflow, especialmente voltados para o processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos conceituais sobre o framework Scikit-Learn

O Scikit-Learn é um dos principais frameworks de uso geral para aprendizado de máquina. Ele possui código aberto e foi desenvolvido em Python. Entre suas funcionalidades, estão aquelas voltadas para processamento de linguagem natural. Esse framework é bastante utilizado para tarefas como classificação de texto, análise de sentimentos e extração de recursos. Para obter informações adicionais e acesso, visite o site oficial em <https://scikit-learn.org/stable/>.

Um dos grandes motivos para a popularidade do Scikit-Learn é a ampla documentação e muitos exemplos disponíveis, o que facilita seu estudo e uso prático. Entre suas muitas características importantes, podemos destacar algumas voltadas para o processamento de linguagem natural. Veja!

1

Extração de recursos

Tem como objetivo capturar as principais características de dados de texto para modelos de aprendizado de máquina.

2

Classificação de texto

Inclui funcionalidades para classificação de texto, como análise de sentimento, detecção de spam e categorização de tópicos.

3 Seleção e avaliação de modelos

Fornecer funções para dividir dados em conjuntos de treinamento e teste, além de também oferecer técnicas para avaliar o desempenho dos modelos.

4

Integração com bibliotecas de PLN

Pode ser integrado com outras bibliotecas e frameworks especializados para processamento de linguagem natural.

5

Eficiência

Possui algoritmos eficientes para problemas de PLN de pequena e média escala.

Em seguida, vamos conhecer um framework muito importante voltado para aplicações de aprendizado de máquina: o Tensorflow.

Aspectos conceituais sobre o framework Tensorflow

O TensorFlow é o principal framework voltado para o aprendizado de máquina que existe atualmente. Ele possui código aberto e foi desenvolvido pelo Google. Entre os diversos domínios de aplicação que podemos utilizá-lo estão os que são voltados para processamento de linguagem natural. Para informações adicionais e acesso, visite o site oficial em <https://www.tensorflow.org/?hl=pt-br>.

Esse framework se destaca de várias maneiras em relação aos outros que vimos até aqui pois, além de possuir diversos algoritmos e modelos eficientes para aplicações de aprendizado de máquina de um modo geral, ele oferece diversos recursos computacionais de gerenciamento de hardware como o uso de **GPU** e **TPUs**, que tornam as soluções muitíssimo eficientes.

GPU, TPUs

GPUs foram inicialmente projetadas para renderizar gráficos em jogos e aplicações gráficas. No entanto, devido à sua arquitetura paralela e capacidade de executar múltiplas tarefas em paralelo, tornaram-se fundamentais no campo de aprendizado de máquina e deep learning. As GPUs são altamente eficientes para acelerar operações matriciais, que são comuns em algoritmos de aprendizado profundo. TPUs são uma classe de aceleradores de hardware desenvolvidos pelo Google especificamente para o processamento de tensores, que são as principais unidades de dados em algoritmos de aprendizado de máquina.

A tarefa de destacar apenas algumas das principais características do Tensorflow não é trivial, devido à amplitude de recursos, ferramentas e algoritmos que possui. Ainda assim, podemos destacar o uso prático dele, que ocorreu por vários motivos, principalmente por sua rica documentação, que conta com diversos exemplos. Além disso, a comunidade de desenvolvedores que compartilham seus projetos no TensorFlow é muito grande e ativa.

Atividade 4

O framework Scikit-Learn oferece diversos recursos para o desenvolvimento de aplicações de aprendizado de máquina de um modo geral. Dessa forma, ele não fica limitado apenas a soluções na área de processamento de linguagem natural. Nesse sentido, selecione a opção que apresente corretamente o principal objetivo principal do framework Scikit-Learn.

A

Criação de interfaces gráficas interativas baseadas na experiência do usuário.

B

Processamento e análise de imagens em tarefas de visão computacional.

C

Construir, treinar e avaliar modelos de aprendizado de máquina.

D

Simulação de diálogo entre homem e máquina.

E

Desenvolvimento de videogames com gráficos avançados.



A alternativa C está correta.

O Scikit-learn é uma biblioteca gratuita, de código aberto, para *machine learning* (aprendizado de máquina) em Python. Ela também fornece uma seleção de recursos eficientes para modelagem estatística, análise e mineração de dados, além de suporte ao aprendizado supervisionado e não supervisionado.

O framework Transformer

Assista ao vídeo para obter uma análise completa dos principais aspectos do framework Transformer, com ênfase em suas principais funcionalidades no contexto do processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos conceituais sobre o framework Transformer

O framework Transformer utiliza uma das arquiteturas de aprendizagem profunda mais modernas que existe. Ele foi apresentado no artigo *Attention Is All You Need* (Vaswani *et al.* 2017). Para mais informações visite o site oficial em <https://huggingface.co/docs/transformers/index>.

De forma resumida, esse framework ganhou destaque no processamento de linguagem natural devido à capacidade de lidar com dados sequenciais de forma eficiente, capturar dependências de longo alcance e facilitar a paralelização. Em seguida, confira algumas de suas principais características.

1

Mecanismo de atenção

Utiliza o mecanismo de atenção, que permite ao modelo fazer a ponderação entre a importância de diferentes palavras em uma frase ao gerar uma determinada palavra.

2 Paralelização

Permite o processamento paralelo de palavras, tornando o processo de treinamento mais rápido em comparação com modelos que dependem de processamento sequencial.

3

Contexto bidirecional

Realiza o processamento dos textos nas duas direções simultaneamente.

4

Modelos pré-treinados

Utiliza modelos pré-treinados avançados, como BERT (representações de codificador bidirecional de transformadores), GPT (transformador generativo pré-treinado) e T5 (transformador de transferência de texto para texto) que, atualmente, estão no estado da arte do processamento de linguagem natural.

Além de todas essas características, também podemos destacar a aprendizagem por transferência entre modelos pré-treinados e sua capacidade de se adaptar a diferentes idiomas e capturar relações linguísticas complexas.

Na sequência, vamos conhecer algumas das principais funcionalidades do framework Transformer voltadas para processamento de linguagem natural.

Principais funcionalidades de PLN do framework Transformer

O framework Transformer oferece diversos recursos para processamento de linguagem natural. Vamos apresentar agora alguns desses recursos com trechos de exemplos.

Classificação de texto

Tem como objetivo categorizar o texto em classes ou rótulos pré-definidos. Vejamos um exemplo usando o modelo BERT.

python

```
from transformers import BertForSequenceClassification, BertTokenizer
modelo = BertForSequenceClassification.from_pretrained('bert-base-uncased')
tokenizador = BertTokenizer.from_pretrained('bert-base-uncased')
entrada = tokenizador("Eu trabalho com Transformers para PLN!", return_tensors="pt")
saida = modelo(**entrada)
```

Reconhecimento de entidades nomeadas

Tem como objetivo identificar e classificar entidades nomeadas em texto. O trecho de código a seguir apresenta como utilizar o reconhecimento de entidades nomeadas com Transformer.

python

```
from transformers import BertForTokenClassification, BertTokenizer
modelo = BertForTokenClassification.from_pretrained('dbmdz/bert-large-cased-finetuned-conll103-english')
tokenizador = BertTokenizer.from_pretrained('bert-base-uncased')
entrada = tokenizador("Cada vez mais o estudo de PLN será um grande diferencial.",
return_tensors="pt")
saida = modelo(**entrada)
```

Resposta a perguntas

Tem como objetivo extrair respostas de determinado contexto para questões específicas. O próximo trecho de código apresenta como utilizar o framework Transformer para trabalhar com respostas a perguntas.

python

```
from transformers import DistilBertForQuestionAnswering, DistilBertTokenizer
modelo = DistilBertForQuestionAnswering.from_pretrained('distilbert-base-cased-distilled-squad')
tokenizador = DistilBertTokenizer.from_pretrained('distilbert-base-cased-distilled-squad')
questao = "Qual o uso do framework Transformers?"
contexto = "O framework Transformers é usado para tarefas de processamento de linguagem natural."
entrada = tokenizador(questao, contexto, return_tensors="pt")
saida = modelo(**entrada)
```

Vimos apenas alguns exemplos do que podemos fazer com um framework voltado para processamento de linguagem natural. Certamente, o potencial de aplicações é muito grande e, por isso mesmo, precisamos consolidar os nossos conhecimentos. Agora, chegou a hora de fazermos mais um exercício. Bons estudos!

Atividade 5

O framework Transformer é considerado o estado da arte para aplicações de processamento de linguagem natural. Ele nos permite realizar diversas aplicações muito sofisticadas, que são úteis para tarefas do dia a dia. Nesse sentido, selecione a opção que apresente corretamente os objetivos básicos do framework Transformer para o desenvolvimento de aplicações de processamento de linguagem natural.

A

Criação de interfaces de usuário simples e interativas.

B

Simulação do comportamento de diversos modelos de aprendizado de máquina.

C

Análise e processamento de imagens para tarefas de visão computacional.

D

Desenvolvimento de jogos voltados para educação.

E

Geração de texto, tradução e análise de sentimento.



A alternativa E está correta.

Um modelo transformer é uma rede neural que aprende o contexto e, assim, o significado com o monitoramento de relações em dados sequenciais como as palavras desta frase. Transformers estão traduzindo texto e fala quase em tempo real, possibilitando que participantes diversos e com deficiências auditivas participem de reuniões e salas de aula.

Exemplos de PLN com SpaCy

Assista ao vídeo e aprenda a demonstrar seu conhecimento na aplicação do framework spaCy para desenvolver aplicações de PLN.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Aqui, você deve aplicar as técnicas de tokenização em um texto, reconhecimento de parte da fala (discurso) e reconhecimento de entidades nomeadas, por meio do framework spaCy usando o Python. Para alcançar esse objetivo, siga os seguintes passos.

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código.

```
python
!pip install spacy tensorflow
```

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código.

```
python
!python -m spacy download pt_core_news_sm
```

Não esqueça: o aninhamento do código faz parte da sintaxe do Python. Se o código escrito por você não estiver corretamente aninhado, seu programa não vai executar.

Passo 5

Adicione uma nova célula de código. Escreva e execute, exatamente, o seguinte código.

```
python

import spacy

# Carregue o modelo spaCy NLP para o português do Brasil
nlp = spacy.load('pt_core_news_sm')

# Exemplo de texto para processamento (português)
texto = "No Brasil, existem pessoas amigáveis e inteligentes que gostam de estudar PLN."

# Processar o texto usando spaCy
documento = nlp(texto)
#documento = texto
# Tokenização
print("Tokens:")
for token in documento:
    print(token.text)

# Parte da Fala
print("\nParte da Fala:")
for token in documento:
    print(token.text, token.pos_)

# Reconhecimento de Entidade Nomeada
print("\nReconhecimento de Entidade Nomeada:")
for entidade in documento.ents:
    print(entidade.text, entidade.label_)
```

Dessa forma, o resultado da execução será:


```
python
```

```
Tokens:
```

```
No
```

```
Brasil
```

```
,
```

```
existem
```

```
pessoas
```

```
amigáveis
```

```
e
```

```
inteligentes
```

```
que
```

```
gostam
```

```
de
```

```
estudar
```

```
PLN
```

```
.
```

```
Parte da Fala:
```

```
No ADP
```

```
Brasil PROP
```

```
, PUNCT
```

```
existem VERB
```

```
pessoas NOUN
```

```
amigáveis ADJ
```

```
e CONJ
```

```
inteligentes ADJ
```

```
que PRON
```

```
gostam VERB
```

```
de CONJ
```

```
estudar VERB
```

```
PLN PROP
```

```
. PUNCT
```

```
Reconhecimento de Entidade Nomeada:
```

```
Brasil LOC
```

O resultado da execução do nosso código mostra o potencial do que podemos realizar com o framework spaCy. Agora, é a sua vez de fazer um exercício que vai lhe ajudar a consolidar seus conhecimentos. Bons Estudos!

Atividade 1

Usando o exemplo da prática, identifique o trecho de código

```
python
```

```
documento = nlp(texto)
```

Agora, crie uma outra célula de código e execute o trecho apresentado a seguir:

```
python
```

```
type(documento)
```

Qual é o resultado exibido?

Chave de resposta

O resultado exibido é:

```
python  
spacy.tokens.doc.Doc
```

Isso significa que um documento é um objeto reconhecido pelo spaCy.

Exemplos de PLN com Gensim

Explore no vídeo a oportunidade para demonstrar seu conhecimento na aplicação do framework Gensim no desenvolvimento de aplicações de PLN.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Aqui, você deve fazer o download de dados de texto das obras do autor brasileiro Machado de Assis, que estão disponíveis na biblioteca NLTK. Em seguida, vamos realizar um teste de semelhança entre uma palavra específica com outras palavras no *corpus* de texto que baixamos por meio do framework Gensim usando o Python. Para alcançar esse objetivo, siga os seguintes passos.

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código.

```
python  
pip install gensim
```

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código.

```
python
!pip install nltk
```

Passo 5

Crie uma nova célula de código. Escreva e execute, exatamente, o seguinte código.

```
python
import nltk
nltk.download('punkt')
```

Não esqueça: o aninhamento do código faz parte da sintaxe do Python. Se o código escrito por você não estiver corretamente aninhado, seu programa não vai executar.

Passo 6

Adicione uma nova célula de código. Escreva e execute, exatamente, o código a seguir.

```
python
import gensim
from gensim.models import Word2Vec
import nltk
from nltk.corpus import machado

# Download dos dados do NLTK sobre Machado de Assis
nltk.download('machado')

# Carregar o corpus sobre Machado de Assis
sentences = machado.sents()

# Treinar o modelo Word2Vec
model = Word2Vec(sentences, vector_size=100,
                  window=5, min_count=1,
                  workers=4)

# Testar a semelhança de uma palavra com outras
word = "amor"
similar_words = model.wv.most_similar(word)

print(f"Incorporações de Palavras para '{word}':")
for similar_word, similarity in similar_words:
    print(f"{similar_word}: {similarity:.4f}")
```

Dessa forma, o resultado da execução será:

```
python
```

```
[nltk_data] Downloading package machado to /root/nltk_data...  
Incorporações de Palavras para 'amor':  
sentimento: 0.8477  
talento: 0.8107  
interesse: 0.7835  
caráter: 0.7709  
destino: 0.7597  
ódio: 0.7544  
respeito: 0.7533  
gênio: 0.7506  
valor: 0.7503  
ofício: 0.7502
```

O resultado da execução do nosso código mostra a semelhança da palavra amor com algumas palavras no *corpus* de texto que baixamos. Esse tipo de aplicação é bastante útil para procurarmos variações de palavras. Assim, temos uma boa ideia do potencial do que podemos realizar com o framework Gensim. Agora, vamos fazer mais um exercício que vai ajudar a consolidar os conhecimentos adquiridos até aqui.

Atividade 2

Usando o exemplo da prática, identifique o trecho de código:

```
python
```

```
word = "amor"
```

Agora, modifique para:

```
python
```

```
word = "transporte"
```

Execute o código. Qual é o resultado exibido?

Chave de resposta

O resultado exibido é:

```
python
```

```
Incorporações de Palavras para 'transporte':
```

```
salvamento: 0.9170
```

```
manto: 0.9170
```

```
cárcere: 0.9104
```

```
mecanismo: 0.9030
```

```
cortejo: 0.8972
```

```
granito: 0.8929
```

```
ninho: 0.8926
```

```
tribunal: 0.8924
```

```
ruído: 0.8910
```

```
Bonaparte: 0.8902
```

Isso significa o grau de semelhança da palavra **transporte** com outras que estão no *corpus* de texto que baixamos da obra de Machado de Assis.

Análise de sentimentos com uma RNN usando Scikit-Learn

Assista ao vídeo para uma introdução sobre o emprego de uma rede neural recorrente com o framework Scikit-Learn na análise de sentimentos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Aqui, você deve aplicar as técnicas de análise de sentimentos usando um tipo de rede neural recorrente (RNN) por meio do framework Scikit-Learn usando o Python. Para alcançar esse objetivo, siga os seguintes passos.

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código.

```
python
```

```
!pip install scikit-learn pandas tensorflow
```

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o código a seguir.

```

python

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

# Amostra de dados para análise de sentimentos
data = [
    ("O filme é maravilhoso! Adorei.", 1),
    ("O enredo estava confuso, e a atuação não foi boa.", 0),
    ("Este restaurante tem a melhor comida da cidade.", 1),
    ("O serviço foi terrível, e o lugar estava sujo.", 0)
]

# Converter dados para um DataFrame
df = pd.DataFrame(data, columns=['text', 'sentiment'])

# Tokenização e ajustes
tokenizer = Tokenizer()
tokenizer.fit_on_texts(df['text'])
sequences = tokenizer.texts_to_sequences(df['text'])
word_index = tokenizer.word_index
max_length = max(len(sequence) for sequence in sequences)
data_padded = pad_sequences(sequences, maxlen=max_length)

# Codificação para rótulos
encoder = LabelEncoder()
labels_encoded = encoder.fit_transform(df['sentiment'])
labels_one_hot = to_categorical(labels_encoded)

# Separação de dados de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(data_padded, labels_one_hot,
test_size=0.25, random_state=42)

# Criar e compilar o modelo RNN
model = Sequential()
model.add(Embedding(input_dim=len(word_index) + 1, output_dim=64,
input_length=max_length))
model.add(LSTM(64))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(2, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# Treinar o modelo
model.fit(X_train, y_train, epochs=10, batch_size=2)

# Avaliar o modelo
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

```

Não esqueça: o aninhamento do código faz parte da sintaxe do Python. Se o código escrito por você não estiver corretamente aninhado, seu programa não vai executar.

Passo 5

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código.

```
python

# Testar uma frase
test_sentence = "O serviço não foi bom."

# Tokenizar e ajustar a frase de testes
test_sequence = tokenizer.texts_to_sequences([test_sentence])
test_padded = pad_sequences(test_sequence, maxlen=max_length)

# Fazer a classificação
prediction = model.predict(test_padded)[0]
sentiment_label = "positivo" if prediction[1] > prediction[0] else "negativo"
confidence = max(prediction) * 100

print(f"Frase de Teste: '{test_sentence}')"
print(f"Sentimento: {sentiment_label}")
print(f"Confiança: {confidence:.2f}%")
```

Dessa forma, o resultado da execução será:

```
python

Frase de Teste: 'O serviço não foi bom.'
Sentimento: positivo
Confiança: 52.68%
```

O resultado da execução do nosso código mostra o potencial do que podemos realizar com o framework Scikit-Learn com RNNs.

Agora, é a sua vez de fazer um exercício que vai lhe ajudar a consolidar seus conhecimentos. Bons Estudos!

Atividade 3

Usando o exemplo da prática, identifique o trecho de código:

```
python

test_sentence = "O serviço não foi bom."
```

Agora, modifique o código da seguinte forma:

```
python

test_sentence = "A comida estava deliciosa, e o atendimento foi ótimo."
```

Qual é o resultado exibido?

Chave de resposta

O resultado obtido foi:

```
python
```

```
Frase de Teste: 'A comida estava deliciosa, e o atendimento foi ótimo.'
```

```
Sentimento: positivo
```

```
Confiança: 52.16%
```

A precisão dos resultados está relacionada ao conjunto de dados que fornecemos para o modelo. É essencial levar esse tipo de consideração, quando utilizarmos esses modelos para aplicações reais.

Análise de sentimentos com uma CNN usando Scikit-Learn

Assista ao vídeo e observe a apresentação introdutória do uso de uma rede neural convolucional com o framework Scikit-Learn para análise de sentimentos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro na prática

Aqui, você deve aplicar as técnicas de análise de sentimentos usando um tipo de rede neural de convolução (CNN) por meio do framework Scikit-Learn usando o Python. Apesar do exemplo ser bastante parecido com o que vimos com RNN, é importante lembrarmos que a CNN é usada, normalmente, para aplicações de visão computacional. Agora, vamos implementar a nossa aplicação. Para alcançar esse objetivo, siga os seguintes passos.

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código.

```
python
```

```
!pip install scikit-learn pandas tensorflow
```

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código.


```
python

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

# Amostra de dados para análise de sentimentos
data = [
    ("O filme é maravilhoso! Adorei.", 1),
    ("O enredo estava confuso, e a atuação não foi boa.", 0),
    ("Este restaurante tem a melhor comida da cidade.", 1),
    ("O serviço foi terrível, e o lugar estava sujo.", 0)
]

# Converter dados para um DataFrame
df = pd.DataFrame(data, columns=['text', 'sentiment'])

# Tokenização e ajuste
tokenizer = Tokenizer()
tokenizer.fit_on_texts(df['text'])
sequences = tokenizer.texts_to_sequences(df['text'])
word_index = tokenizer.word_index
max_length = max(len(sequence) for sequence in sequences)
data_padded = pad_sequences(sequences, maxlen=max_length)

# Codificação de rótulos
encoder = LabelEncoder()
labels_encoded = encoder.fit_transform(df['sentiment'])
labels_one_hot = to_categorical(labels_encoded)

# Separação de dados para treinamento e testes
X_train, X_test, y_train, y_test = train_test_split(data_padded, labels_one_hot,
test_size=0.25, random_state=42)

# Criar e compilar modelo CNN
model = Sequential()
model.add(Embedding(input_dim=len(word_index) + 1, output_dim=64,
input_length=max_length))
model.add(Conv1D(128, 5, activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(2, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# Treinamento do modelo
model.fit(X_train, y_train, epochs=10, batch_size=2)

# Avaliar o modelo
loss, accuracy = model.evaluate(X_test, y_test)
print("Teste de Precisão:", accuracy)
```

Não esqueça: o aninhamento do código faz parte da sintaxe do Python. Se o código escrito por você não estiver corretamente aninhado, seu programa não vai executar.

Passo 5

Crie outra célula de código. Escreva e execute, exatamente, o código a seguir.

```
python

# Testar uma frase
test_sentence = "O serviço foi razoável."

# Tokenizar e ajustar uma frase
test_sequence = tokenizer.texts_to_sequences([test_sentence])
test_padded = pad_sequences(test_sequence, maxlen=max_length)

# Fazer a classificação
prediction = model.predict(test_padded)[0]
sentiment_label = "positivo" if prediction[1] > prediction[0] else "negativo"
confidence = max(prediction) * 100

print(f"Frase de Teste: '{test_sentence}')"
print(f"Sentimento: {sentiment_label}")
print(f"Confiança: {confidence:.2f}%")
```

Dessa forma, o resultado da execução será:

```
python

Frase de Teste: 'O serviço foi razoável.'
Sentimento: positivo
Confiança: 62.60%
```

O resultado da execução do nosso código mostra o potencial do que podemos realizar com o framework Scikit-Learn com CNNs.

Agora, é a sua vez de fazer um exercício que vai ajudá-lo a consolidar seus conhecimentos. Bons estudos!

Atividade 4

Usando o exemplo da prática, identifique o trecho de código:

```
python

test_sentence = "O serviço foi razoável."
```

Agora, modifique o código da seguinte forma:

```
python

test_sentence = "A comida estava deliciosa, e o atendimento foi ótimo."
```

Qual é o resultado exibido?

Chave de resposta

O resultado obtido foi:

```
python
```

```
Testa a frase: 'A comida estava deliciosa, e o atendimento foi ótimo.'
```

```
Sentimento: positivo
```

```
Confiança: 61.28%
```

Novamente observamos que a precisão dos resultados está relacionada ao conjunto de dados que fornecemos para o modelo. As redes CNNs têm sido utilizadas com muito sucesso nas aplicações de processamento de linguagem natural.

Análise de sentimentos usando o framework Transformer

Confira no vídeo uma introdução ao uso do framework Transformers para análise de sentimentos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Aqui, você deve aplicar as técnicas de análise de sentimentos usando o framework Transformer. É importante lembrarmos que esse é estado da arte para aplicações de PLN. Agora, vamos implementar a nossa aplicação. Para alcançar esse objetivo, siga os passos a seguir.

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código.

```
python
```

```
!pip install transformers
```

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código.

```
python

from transformers import pipeline

# Carregar o pipeline de análise de sentimentos
nlp = pipeline("sentiment-analysis")

# Texto para análise de sentimentos
text = "I am really enjoying this movie. It's so much fun!"

# Realizar a análise de sentimentos
results = nlp(text)

# Interpretar o rótulo e a pontuação do sentimento
sentiment_label = results[0]['label']
sentiment_score = results[0]['score']

print(f"Texto: {text}")
print(f"Rótulo do Sentimento: {sentiment_label}")
print(f"Pontuação do Sentimento: {sentiment_score:.4f}")
```

Não esqueça: o aninhamento do código faz parte da sintaxe do Python. Se o código escrito por você não estiver corretamente aninhado, seu programa não vai executar.

Dessa forma, o resultado da execução será:

```
python

Texto: I am really enjoying this movie. It's so much fun!
Rótulo do Sentimento: POSITIVE
Pontuação do Sentimento: 0.9999
```

O resultado da execução do nosso código mostra o potencial do que podemos realizar com o framework Transformer. Agora, mais um exercício para ajudar a consolidar seus conhecimentos. Bons estudos!

Atividade 5

Usando o exemplo da prática, identifique o trecho de código:

```
python

text = "I am really enjoying this movie. It's so much fun!"
```

Agora, modifique o código da seguinte forma:

```
python

text = "I didn't like this movie!"
```

Qual é o resultado exibido?

Chave de resposta

O resultado obtido foi:

```
python
```

```
Texto: I didn't like this movie!
```

```
Rótulo do Sentimento: NEGATIVE
```

```
Pontuação do Sentimento: 0.9899
```

No caso, obtivemos um resultado negativo. Conseguimos obter essa classificação porque utilizamos um modelo pré-treinado.

Considerações finais

O que você aprendeu neste conteúdo?

- As principais redes neurais aplicadas para PLN.
- Os principais frameworks computacionais utilizados para PLN.
- Exemplos práticos com Python de aplicações de redes neurais para PLN.

Explore +

Acesse o site oficial da Hugging Face Transformers. Nele, você vai aprofundar seus conhecimentos sobre esse poderoso framework e encontrará diversos exemplos.

Acesse o site oficial do Microsoft Azure e busque por “*conversational language understanding*”. Você vai encontrar aplicações práticas de mercado sobre processamento de linguagem natural.

Referências

BIRD, S.; KLEIN, E.; LOPER, E. **Natural Language Processing with Python**. O'Reilly Media, 2009.

GOLDBERG, Y. **Neural Network Methods for Natural Language Processing**. Synthesis Lectures on Human Language Technologies, v. 10, n. 1, p. 1-309, 2017.

JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition. 3rd ed. Pearson Education, 2019.

MANNING, C. D.; SCHÜTZE, H. **Foundations of Statistical Natural Language Processing**. MIT Press, 1999.

RASCHKA, S.; MIRJALILI, V. **Python Machine Learning** (3rd ed.). Packt Publishing, 2019.

VASWANI, A.; *et al.* **Attention is All You Need**. In 31st Conference on Neural Information Processing Systems – NIPS. Long Beach, CA, USA, 2017.